NYC Taxi Trip using Amazon Redshift

- Submitted By : **Parth Shah**

---

1. Introduction

The New York City Taxi Trip dataset contains detailed trip-level information for Yellow and Green taxis in New York City, including pickup/drop-off timestamps, locations, fare details, passenger counts, and payment types.

This assignment focuses on implementing data engineering and analytics workflows using:

- Amazon S3 for data storage
- Amazon Redshift Spectrum for external querying
- Amazon Redshift internal tables for optimized storage
- AWS Glue Data Catalog for metadata management
- Amazon RDS PostgreSQL for federated queries

The objective is to analyze query performance and understand how table design affects execution efficiency.

---

2. Dataset Description

- Dataset Source: NYC Taxi & Limousine Commission (TLC)
- Taxi Types:
    - Yellow Taxi
    - Green Taxi
- Time Period: January 2023 – June 2023
- Format: Parquet
- Storage Location: Amazon S3

Key Columns:

- tpep_pickup_datetime
- tpep_dropoff_datetime

- passenger_count
- trip_distance
- fare_amount
- payment_type
- pickup_location_id
- dropoff_location_id

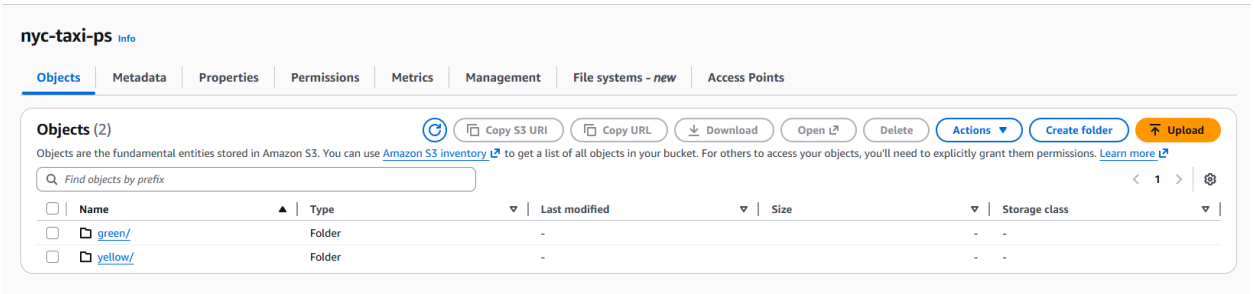
---

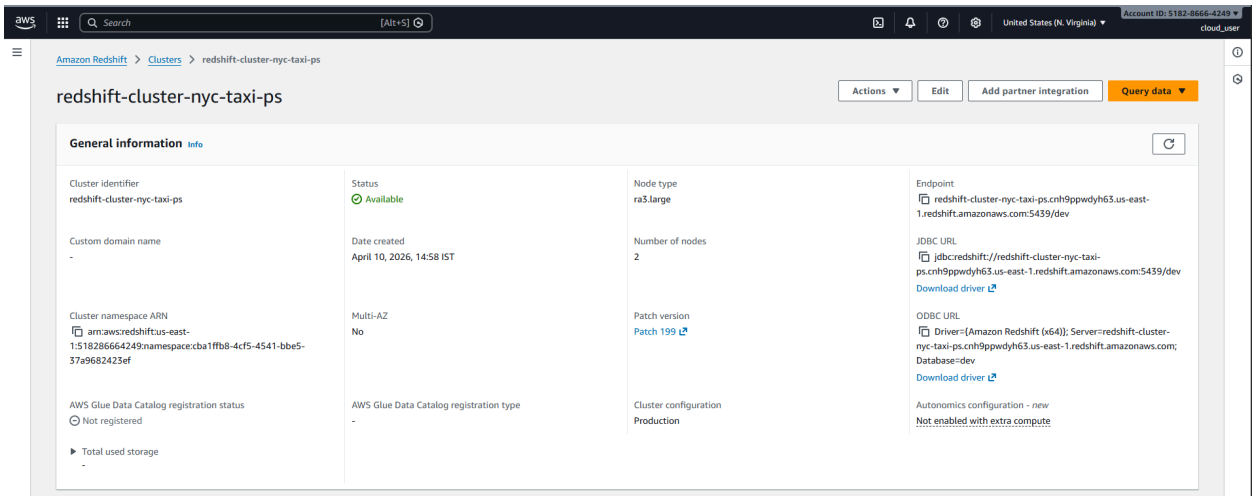
3. Task 1: Redshift Spectrum (External Tables)

3.1 S3 Data Upload

Data was uploaded to Amazon S3 in the following structure:

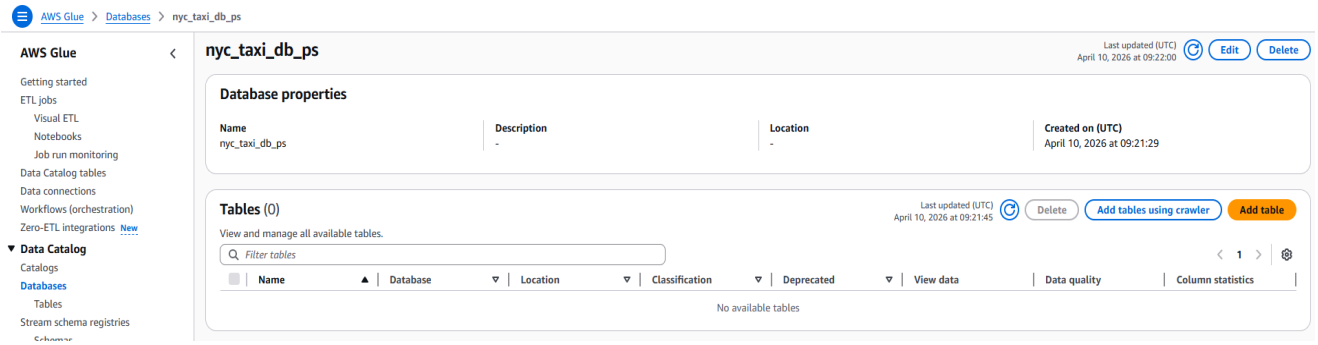
- s3://nyc-taxi-ps/yellow/
- s3://nyc-taxi-ps/green/



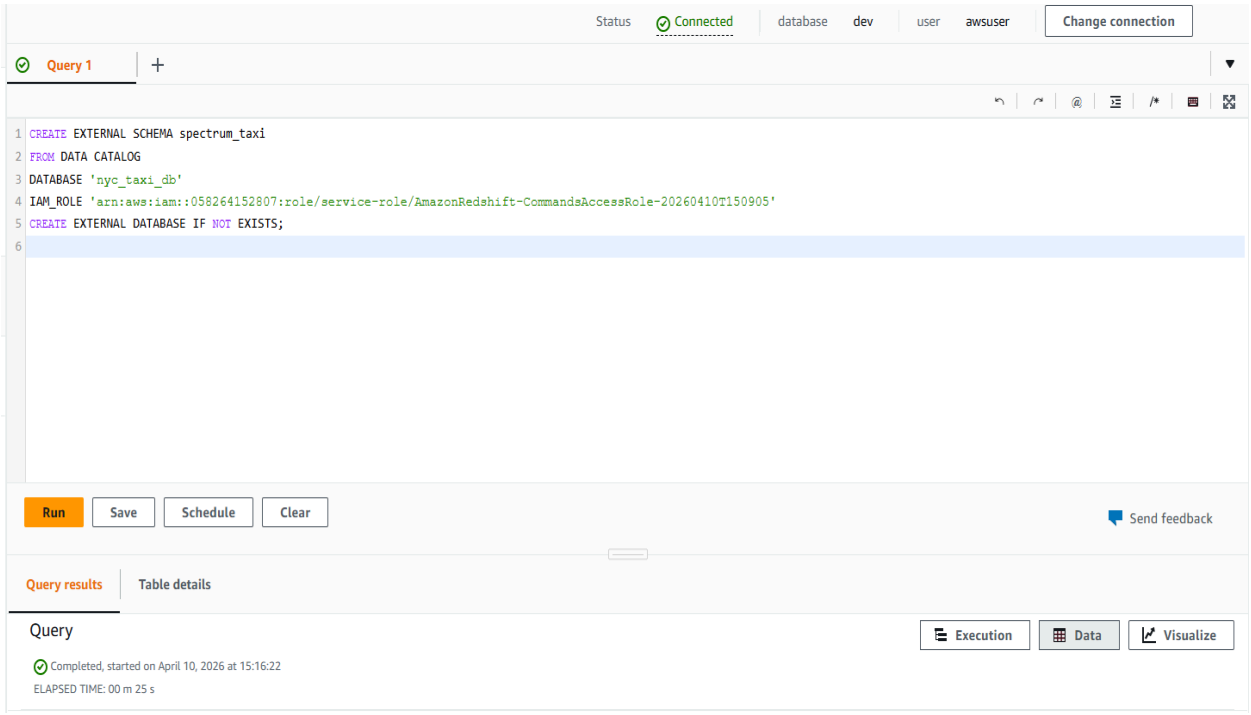


3.2 AWS Glue Data Catalog Setup

- Database Name: `nyc_taxi_db`

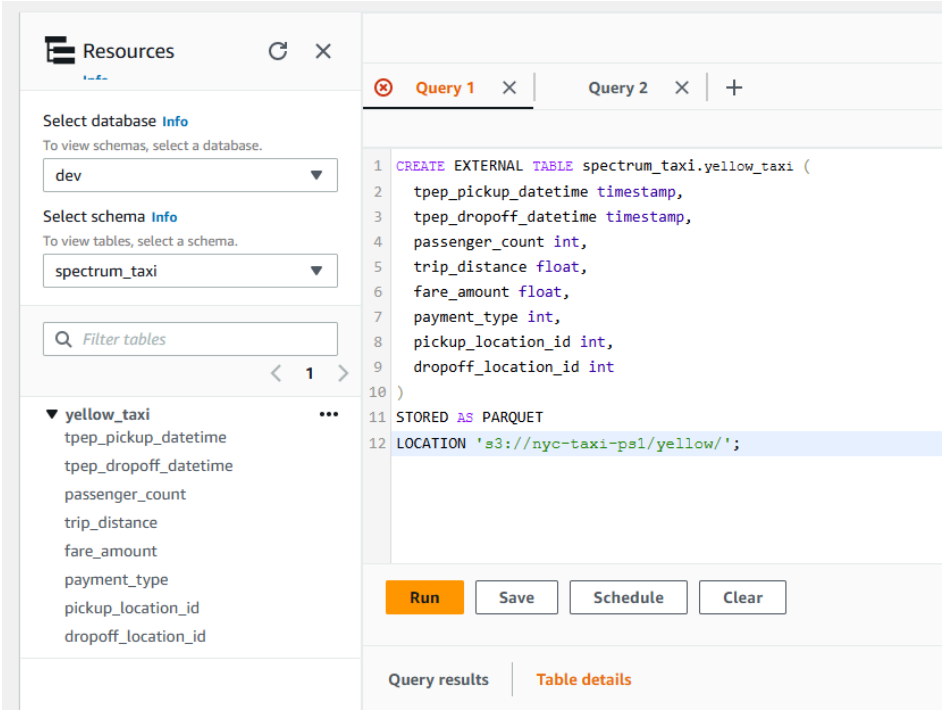


3.3 External Schema Creation

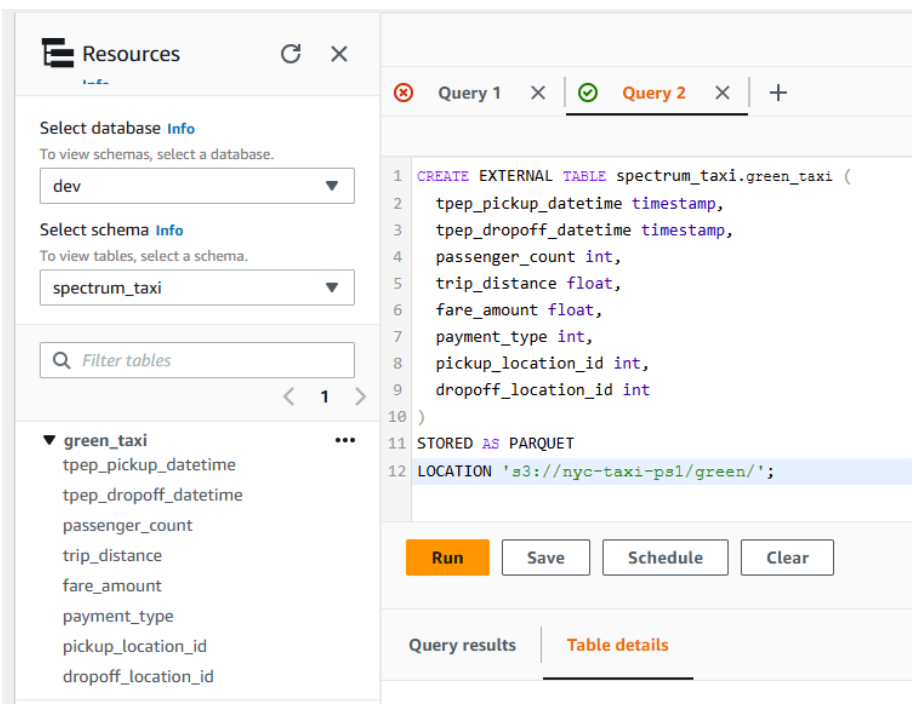


3.4 External Table Definitions

Yellow Taxi External Table



Green Taxi External Table



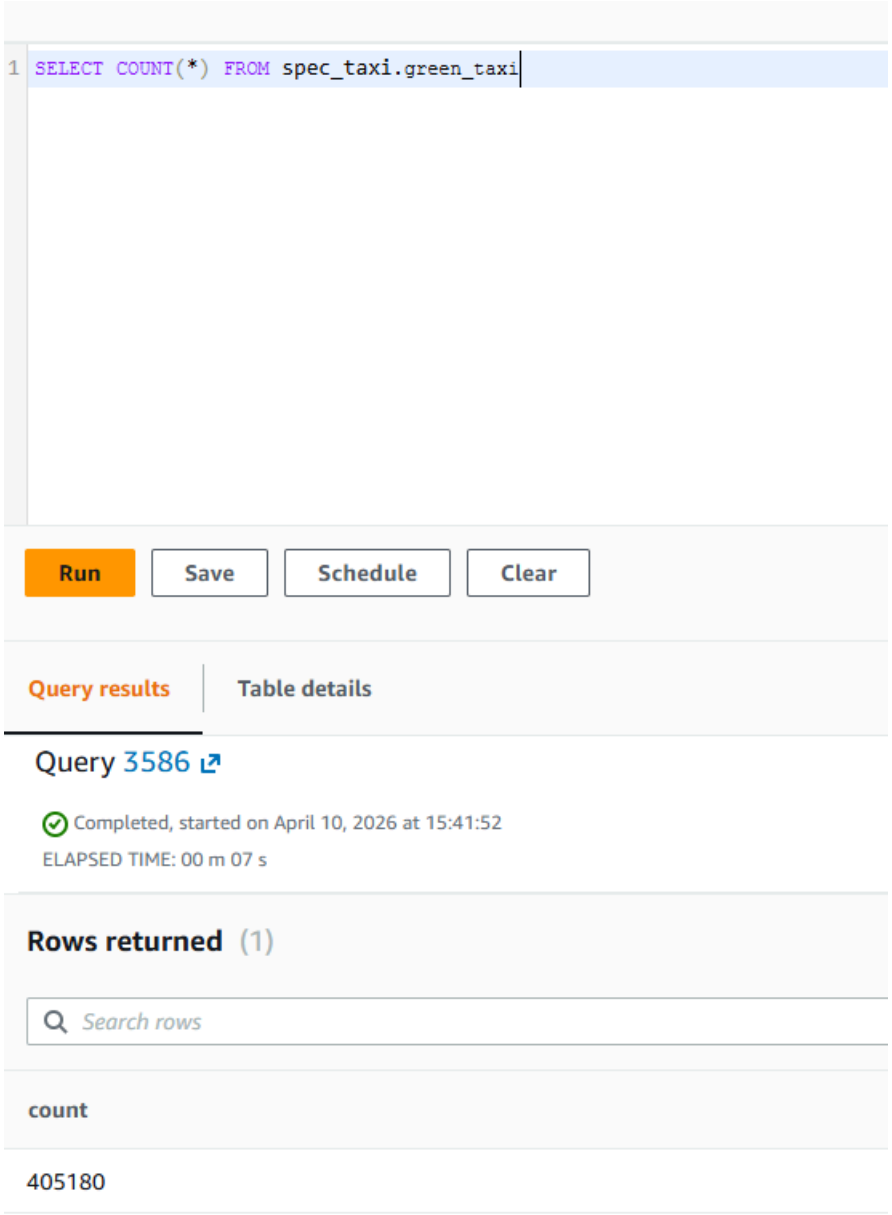
3.5 Spectrum Query Execution Results

Sample Query:

SELECT COUNT(*) FROM spec_taxi.green_taxi

Output:

- Result



4. Task 2: Analytical Queries

4.1 Total Trips per Day by Payment Type & Taxi Type

```
SELECT DATE(tpep_pickup_datetime) AS trip_date,
  payment_type, 'Yellow' AS taxi_type,
  COUNT(*) AS total_trips
FROM spec_taxi.yellow_taxi
GROUP BY 1,2;
```

Query results | Table details

Query 4353

Completed, started on April 10, 2026 at 16:42:01
ELAPSED TIME: 00 m 08 s

Execution | Data | Visualize

Rows returned (931)

Export

Search rows

1 2 3 4 5 6 7 ... 94

trip_date	payment_type	taxi_type	total_trips
2023-01-06	0	Yellow	2279
2023-01-14	0	Yellow	2333
2023-01-23	0	Yellow	2120
2023-01-22	0	Yellow	2125
2023-01-15	0	Yellow	2227
2023-01-09	2	Yellow	15647
2023-01-04	2	Yellow	19207
2023-01-12	4	Yellow	1131
2023-01-20	4	Yellow	1117
2023-01-02	2	Yellow	15159

4.2 Top 10 Trips by Fare Amount

```
SELECT 'yellow'as taxi_type ,pulocationid,dolocationid, fare_amount
FROM spec_taxi.yellow_taxi
ORDER BY fare_amount DESC
LIMIT 10;
```

Result:

Rows returned (10)

Export

Search rows

1

taxi_type	pulocationid	dolocationid	fa
yellow	100	50	386983.63
yellow	239	264	6300.9
yellow	216	265	2449.5
yellow	230	264	2203.1
yellow	264	264	2100
yellow	264	264	1968.75
yellow	163	265	1701.2
yellow	132	265	1160.1
yellow	28	211	1055.5
yellow	132	132	1025

4.3 Average Trip Distance (>2 Passengers)

```
SELECT taxi_type, AVG(trip_distance) AS avg_distance
FROM (
   SELECT 'Yellow' AS taxi_type, trip_distance, passenger_count
   FROM spec_schema.yellow_tripdata

   UNION ALL

   SELECT 'Green', trip_distance, passenger_count
   FROM spectrum_schema.green_tripdata
)
WHERE passenger_count > 2
GROUP BY taxi_type;
```

Result:

taxi_type	avg_distance
Yellow	3.9218061655114917
Green	3.13905503292567

5. Task 3: Load Data into Amazon Redshift

5.1 Internal Table Creation (Unoptimized)

Yellow Taxi Table

```
CREATE TABLE yellow_taxi_internal (
   VendorID INT,
   pickup_datetime TIMESTAMP,
   dropoff_datetime TIMESTAMP,
   passenger_count INT,
   trip_distance FLOAT,
   RatecodeID INT,
   PULocationID INT,
   DOLocationID INT,
   payment_type INT,
   fare_amount FLOAT
);
```

```sql
CREATE TABLE yellow_taxi_internal (
    VendorID INT,
    pickup_datetime TIMESTAMP,
    dropoff_datetime TIMESTAMP,
    passenger_count INT,
    trip_distance FLOAT,
    RatecodeID INT,
    PULocationID INT,
    DOLocationID INT,
    payment_type INT,
    fare_amount FLOAT
);
```

Run | Save | Schedule | Clear

Send feedback

Query results | Table details

Query | Execution | Data | Visualize

Completed, started on April 13, 2026 at 10:50:09

ELAPSED TIME: 00 m 03 s

Green Taxi Table

Query 1 +

```sql
CREATE TABLE public.green_taxi_internal (
    vendorid BIGINT,
    lpep_pickup_datetime TIMESTAMP,
    lpep_dropoff_datetime TIMESTAMP,
    passenger_count BIGINT,
    trip_distance DOUBLE PRECISION,
    ratecodeid BIGINT,
    store_and_fwd_flag VARCHAR(10),
    pulocationid BIGINT,
    dolocationid BIGINT,
    payment_type BIGINT,
    fare_amount DOUBLE PRECISION,
    extra DOUBLE PRECISION,
    mta_tax DOUBLE PRECISION,
    tip_amount DOUBLE PRECISION,
    tolls_amount DOUBLE PRECISION,
    improvement_surcharge DOUBLE PRECISION,
    total_amount DOUBLE PRECISION,
```

Run | Save | Schedule | Clear

Query results | Table details

Query

Completed, started on April 13, 2026 at 12:01:05

ELAPSED TIME: 00 m 11 s

5.2 Data Loading using COPY Command

```
COPY public.yellow_taxi_internal
FROM 's3://nyc-taxi-ps/yello/'
IAM_ROLE
'arn:aws:iam::381492274790:role/AmazonRedshift-CommandsAccessRole-20260413T095143'
FORMAT AS PARQUET;
```

```
COPY public.green_taxi_internal
FROM 's3://nyc-taxi-ps/green/'
IAM_ROLE
'arn:aws:iam::381492274790:role/AmazonRedshift-CommandsAccessRole-20260413T095143'
FORMAT AS PARQUET;
```

5.3 Data Validation

- Row count (Yellow):

---

6. Task 4: Federated Query (Amazon RDS PostgreSQL)

6.1 External PostgreSQL Table (RDS)

Table Name: `payment_lookup`

-- Table creation SQL

```
~ $  psql -h database-2.cjmemcak6h51.us-east-1.rds.amazonaws.com -U postgres -d postgres
Password for user postgres:
psql (15.16, server 17.6)
WARNING: psql major version 15, server major version 17.
         Some psql features might not work.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

postgres=> CREATE TABLE payment_lookup (
postgres(>     payment_type INT PRIMARY KEY,
postgres(>     description VARCHAR(50)
postgres(> );
CREATE TABLE
postgres=> INSERT INTO payment_lookup VALUES
postgres-> (1, 'Credit Card'),
postgres-> (2, 'Cash'),
postgres-> (3, 'No Charge'),
postgres-> (4, 'Dispute'),
postgres-> (5, 'Unknown'),
postgres-> (6, 'Voided Trip');
INSERT 0 6
postgres=> SELECT * FROM payment_lookup;
 payment_type | description
--------------+-------------
            1 | Credit Card
            2 | Cash
            3 | No Charge
            4 | Dispute
            5 | Unknown
            6 | Voided Trip
(6 rows)
```

6.2 Redshift Federated Schema Setup

```
CREATE EXTERNAL SCHEMA rds_schema
FROM POSTGRES
DATABASE 'postgres'
SCHEMA 'public'
URI 'database-2.cjmemcak6h51.us-east-1.rds.amazonaws.com:5432'
IAM_ROLE 'arn:aws:iam::381492274790:role/service-role/AmazonRedshift-CommandsAccessRole-20260413T095143'
SECRET_ARN 'arn:aws:rds:us-east-1:381492274790:db:database-2';
```

Run | Save | Schedule | Clear | Send feedback

Query results | Table details

Query

Execution | Data | Visualize

Completed, started on April 13, 2026 at 13:28:59

ELAPSED TIME: 00 m 11 s

6.3 Federated Query Execution

- SELECT * FROM rds_schema.payment_lookup;

Result:

Rows returned (6)

Export

Search rows

1

payment_type	description
2	Cash
4	Dispute
6	Voided Trip
1	Credit Card
3	No Charge
5	Unknown

7. Task 5: Baseline Performance Analysis (Before Optimization)

7.1 Query Execution Time Summary

Query Name	Execution Time (sec)	Rows Scanned	Data Movement	Observations
Trips per Day (Query 1)	3.1	High	DS_BCAST	Full table scan due to absence of sort key
Top 10 Fares (Query 2)	2.1	Medium	None	Sorting operation on fare_amount column
Avg Distance (>2 passengers)	1.5	High	None	Filter applied but scans large dataset

7.2 Query Execution Plans (Before Optimization)

query	starttime	endtime	execution_time_ms
7718	2026-04-13 09:29:07.164398	2026-04-13 09:29:07.18069	16
7716	2026-04-13 09:28:47.531712	2026-04-13 09:28:47.61648	85
7714	2026-04-13 09:28:47.474649	2026-04-13 09:28:47.490078	16
7712	2026-04-13 09:28:47.427472	2026-04-13 09:28:47.449452	22
7710	2026-04-13 09:28:24.300061	2026-04-13 09:28:24.308561	8

Query 1 Plan:

```
EXPLAIN
SELECT
    DATE(tpep_pickup_datetime),
    COUNT(*)
FROM public.yellow_taxi_internal
GROUP BY 1;
```

Run | Save | Schedule | Clear

Query results | Table details

Query

Completed, started on April 13, 2026 at 15:00:48
ELAPSED TIME: 00 m 01 s

Rows returned (2)

Search rows

QUERYPLAN
XN HashAggregate (cost=329290.61..377456.37 rows=9633153 width=8)
-> XN Seq Scan on yellow_taxi_internal (cost=0.00..235207.58 rows=18816606 width=8)

8. Task 6: Query Optimization

Yellow Taxi Optimized Table

```
CREATE TABLE yellow_taxi_opt
DISTKEY (pulocationid)
SORTKEY (tpep_pickup_datetime)
AS
SELECT * FROM yellow_taxi_internal;
```

Run | Save | Schedule | Clear

Query results | Table details

Query 6815

Completed, started on April 13, 2026 at 14:04:51
ELAPSED TIME: 00 m 29 s

Green Taxi Optimized Table

```
CREATE TABLE green_taxi_opt
DISTKEY (pulocationid)
SORTKEY (lpep_pickup_datetime)
AS
SELECT * FROM green_taxi_internal;
```

Run | Save | Schedule | Clear

Query results | Table details

Query 7101

Completed, started on April 13, 2026 at 14:29:55

ELAPSED TIME: 00 m 04 s

8.2 Re-run Queries (After Optimization)

```
– q1
SELECT
   DATE(tpep_pickup_datetime) AS trip_date,
   COUNT(*) AS total_trips
FROM public.yellow_taxi_optimized
GROUP BY 1
ORDER BY 1;

–q2
SELECT
   payment_type,
   SUM(total_amount) AS total_revenue
FROM public.yellow_taxi_optimized
GROUP BY 1
ORDER BY total_revenue DESC;
```

```
– q3
SELECT
   PULocationID,
   COUNT(*) AS total_trips
FROM public.yellow_taxi_optimized
GROUP BY 1
ORDER BY total_trips DESC
LIMIT 10;
```

query	execution_time_ms
8242	16
8239	594
8235	61

9. Task 7: Performance Comparison & Analysis

9.1 Execution Time Differences :-

Explain improvements observed after optimization:

Table	Column	Encoding	Est_reduction_pct
yellow_taxi_optimized	vendorid	az64	95.35
yellow_taxi_optimized	tpep_pickup_datetime	raw	0.00
yellow_taxi_optimized	tpep_dropoff_datetime	az64	56.41
yellow_taxi_optimized	passenger_count	az64	92.51
yellow_taxi_optimized	trip_distance	zstd	72.18
yellow_taxi_optimized	ratecodeid	az64	92.74
yellow_taxi_optimized	store_and_fwd_flag	zstd	99.70
yellow_taxi_optimized	pulocationid	az64	84.58
yellow_taxi_optimized	dolocationid	az64	83.92
yellow_taxi_optimized	payment_type	az64	93.51

9.2 Query Plan Changes :-

Before Optimization

- Queries performed **sequential scans (Seq Scan)** across the entire table
- High data movement operations:
  - *`DS_BCAST`* (broadcasting data across nodes)
  - *`DS_DIST_ALL`* (full redistribution)
- No effective sorting → inefficient filtering and grouping
- Higher execution cost due to unnecessary data shuffling

After Optimization

- **Reduced data movement** due to proper `DISTKEY`
- Many queries avoided:
  - `DS_BCAST`
  - `DS_DIST_ALL`
- Improved aggregation using **HashAggregate**
- **SORTKEY enabled range-restricted scans** instead of full scans

9.3 Impact of Data Volume :-

Small Data Volume

- Minimal performance difference
- Queries already execute quickly
- Optimization impact is less noticeable

Large Data Volume

- Significant performance improvement observed
- Reduced scan time due to **SORTKEY**
- Reduced data movement due to **DISTKEY**

9.4 Query Optimization Effectiveness

Query Type	Improvement	Reason
GROUP BY	High	Reduced data movement
Time-based filter	High	Efficient scan via SORTKEY
Full table scan	Medium	Still scans entire data
Small dataset queries	Low	Already fast

10. Conclusion

This assignment demonstrated us how data architecture decisions in Amazon Redshift impact query performance. Using Spectrum, federated queries, and optimized internal tables, we analyzed how storage design and distribution strategies significantly affect analytical workloads.